Design and Simulation of an Arduino-Based Automated Plant Growing and Monitoring System

Muhammad Ali Sadiq[**]
School of Engineering, Asia Pacific University of Technology & Innovation (APU), Kuala Lumpur, Malaysia
https://github.com/turtleshelp/automated-plant-monitoring

Course project, August–September 2020

ABSTRACT

This paper presents a low-cost embedded system that monitors the growing conditions of a plant and reacts automatically to keep them in an optimal range. An Arduino Uno reads light intensity, air temperature, relative humidity, soil moisture and the water level of a spray tank. Depending on the readings it switches a grow lamp, a fan, a heat lamp, a water spray and a watering indicator, and it shows every reading and action on a 20×4 character LCD driven over I^2C. The readings are also streamed over a serial link for logging. The system was simulated in Proteus and the light, humidity and water-level functions were tested on hardware. The simulation confirms the light and humidity control rules, and the hardware test reproduces them on a real board. A later review of the firmware identified several defects in the integrated version; these are reported together with the planned improvements.

Keywords: embedded systems, Arduino, plant monitoring, automatic irrigation, Proteus simulation, environmental sensing

I. INTRODUCTION

Plants need suitable light, temperature, humidity and soil moisture to grow well. Checking and correcting these conditions by hand takes time, and it is easy to water too much or too little or to leave a lamp switched on. A microcontroller can measure the conditions continuously and react immediately. This reduces the manual work and helps to avoid wasting water.

The aim of this project was a monitoring and control system built from inexpensive, widely available components that

- measures five environmental quantities around a plant,
- compares each reading with an optimal range and switches the matching actuator,
- displays every reading and action locally and logs the data on a PC.

Section II describes the system design, Section III the firmware, Section IV the simulation and hardware results, and Section V the limitations and planned improvements.

[**]Developed as a group course project (Microprocessor Systems & Embedded Software, APU). The author designed the light, humidity, water-level and soil-moisture subsystems and, together with Nabeel, integrated and simulated the complete system. Nabeel designed the temperature subsystem; Shimal prepared the project documentation.

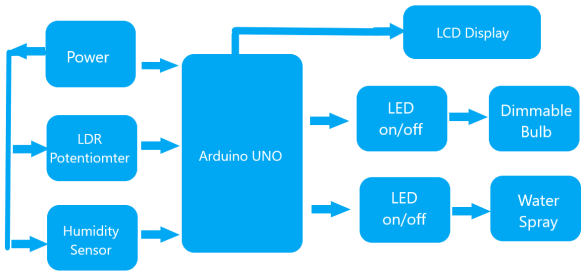


Figure 1: Block diagram of the light and humidity control path. The sensors feed the Arduino Uno, which switches the lamp, the spray and the status LEDs and drives the LCD.

II. SYSTEM DESIGN

A. Architecture

Fig. 1 shows the system architecture. The sensors feed the Arduino Uno, which drives the indicator LEDs and actuators and the LCD. The Arduino Uno is based on the 8-bit ATmega328P microcontroller (16 MHz, 10-bit ADC, 14 digital I/O pins, 6 analogue inputs) [1, 2].

B. Hardware

Sensors. Temperature and humidity are measured with a DHT11 single-wire sensor on D2 [3]. Light intensity is measured with a light-dependent resistor (LDR) on A0. A water-level sensor on A1 monitors the spray tank, and a soil-moisture sensor on A3 measures the water content of the soil.

Outputs. The grow lamp, heat lamp and status indicators are LEDs. In the simulation the fan, the spray and the watering pump are DC-motor models. All outputs use the digital pins D0–D13.

Display. A 20×4 HD44780-compatible LCD [4] is connected through a PCF8574 I^2C port expander [5] at address 0x27. This needs only the two I^2C lines (A4/A5) instead of six or more digital pins. Fig. 2 shows the light and humidity circuit with the I^2C LCD.

C. Control Rules

Each quantity is compared with a lower and an upper threshold (Table 1). Below the range the system corrects in one direction, above it in the other, and inside the range it takes no action. For the LDR divider a higher ADC value means less light.

III. FIRMWARE

The firmware is written in Arduino C and uses the Adafruit DHT library [7] and the LiquidCrystal_I2C library [8].

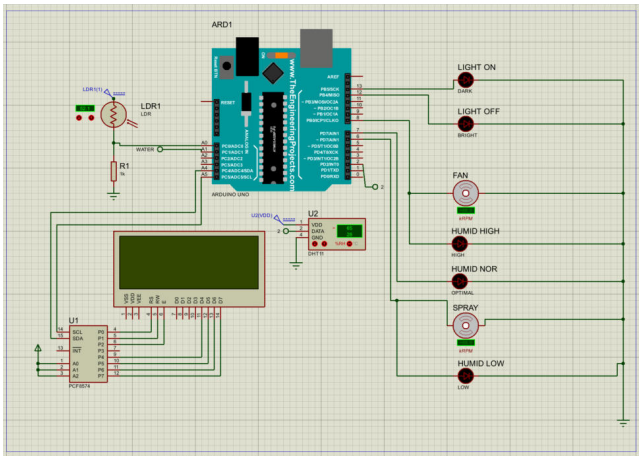


Figure 2: Proteus schematic of the light and humidity subsystem: LDR divider on A0, DHT11 on D2, LCD via PCF8574 on A4/A5, and the status LEDs and fan/spray motors on the digital pins.

Table 1: Control rules

Input	Condition	Action
Light (raw ADC)	$>$ 200 (dark)	Lamp ON
	$<$ 150 (bright)	Lamp OFF
	150–200	No action
Humidity	$>$ 60 %	Fan ON
	$<$ 50 %	Spray ON
Temperature	$>$ 29 °C	Fan ON
	27–29 °C	Normal
	otherwise	Heat lamp ON
Soil moisture	$<$ 20 %	Watering ON
	$>$ 50 %	Watering OFF
Tank level	$<$ 20 %	“LOW” (refill)
	$>$ 80 %	“HIGH”

Fig. 3 shows the control flow for light and humidity. Each `loop()` pass first reads all sensors and then handles the subsystems one after another. For each subsystem it prints the reading and the decision on the LCD, sets the corresponding outputs and holds the screen for 2 s, for example:

```
if (ldrStatus > 200) { // dark
  digitalWrite(ledPin13, HIGH); // lamp on
  lcd.print("ITS DARK");
  ...
  delay(2000);
}
```

The analogue sensors are converted to percentages from the ADC reading. Each reading is also printed over the serial port at 9600 baud as comma-separated values, so it can be logged on a PC.

IV. SIMULATION AND RESULTS

A. Proteus Simulation

The system was simulated in Proteus, which co-simulates the microcontroller firmware with the connected circuit [6]. The compiled firmware (`.hex`) is loaded into the Arduino model, and the LDR light level and the DHT11 readings are changed interactively. Fig. 4 shows the LCD for three light levels and three humidity levels. In each case the displayed decision matches Table 1. A reading of 214 is reported as dark with the lamp switched on, 124 as bright with the lamp

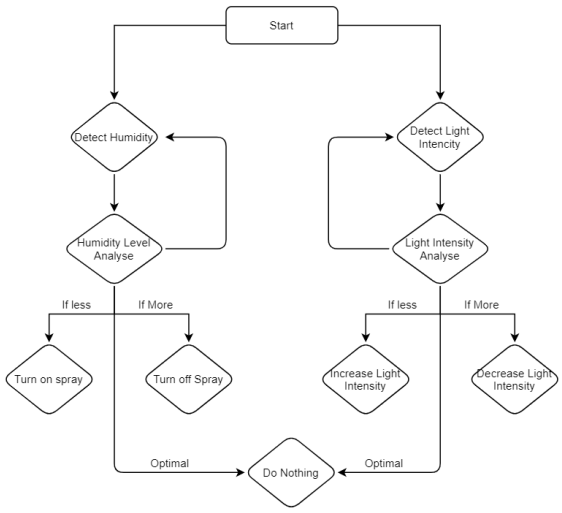


Figure 3: Control flow of the light and humidity subsystems.

off, and 167 as optimal. A humidity of 66 % switches the fan on, 46 % switches the spray on, and 52 % is optimal. The corresponding LEDs and motor models switched with the displayed state. Fig. 5 shows the integrated system with all five sensors.

B. Hardware Test

The firmware was then run on a real Arduino Uno with a DHT11, an LDR, a water-level sensor and a 20×4 I^2C LCD. Fig. 6 shows three of the recorded states. High humidity (73 %) switched the fan on, low light (raw ADC 295) switched the lamp on, and an empty tank (0 %) was reported as low.

C. Data Logging

The serial output was streamed into a spreadsheet with Excel Data Streamer (Fig. 7). Over 15 samples spanning 31 s at room conditions the readings were stable: light 276–280, temperature 28.1–28.2 °C and humidity 76–77 %. The moisture channel, however, logged -85 to -86 %, a physically impossible value that points to the calibration defect discussed in Section V.

V. DISCUSSION AND LIMITATIONS

The simulation and the hardware test confirm the light and humidity control and the water-level display. A later review of the firmware found the following defects, mainly in the integrated version:

- The tank level is computed from the soil sensor’s pin (A3) instead of the water-level sensor (A1), so it repeats the soil reading.
- In the integrated version the humidity thresholds were changed to $>$ 50 % and $<$ 0 %, so the spray can never switch on.
- The soil mapping `map(raw, 550, 10, 0, 100)` is not clamped, so out-of-range readings give negative percentages (the likely cause of the logged -86 %). Its result is also overwritten by a second variable with the same name.
- Exactly 29 °C is classified as “too low” and switches the heat lamp on.

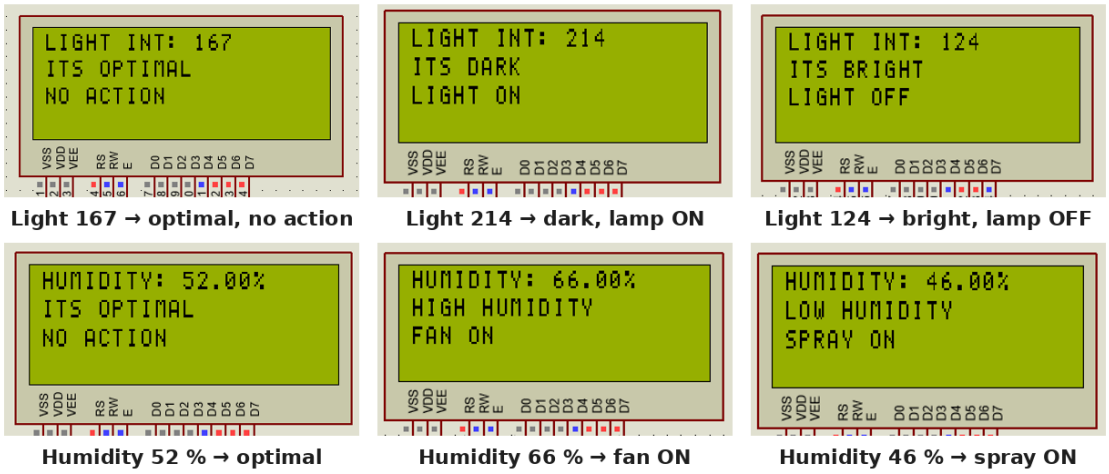


Figure 4: LCD output in the Proteus simulation for three light levels (top) and three humidity levels (bottom).

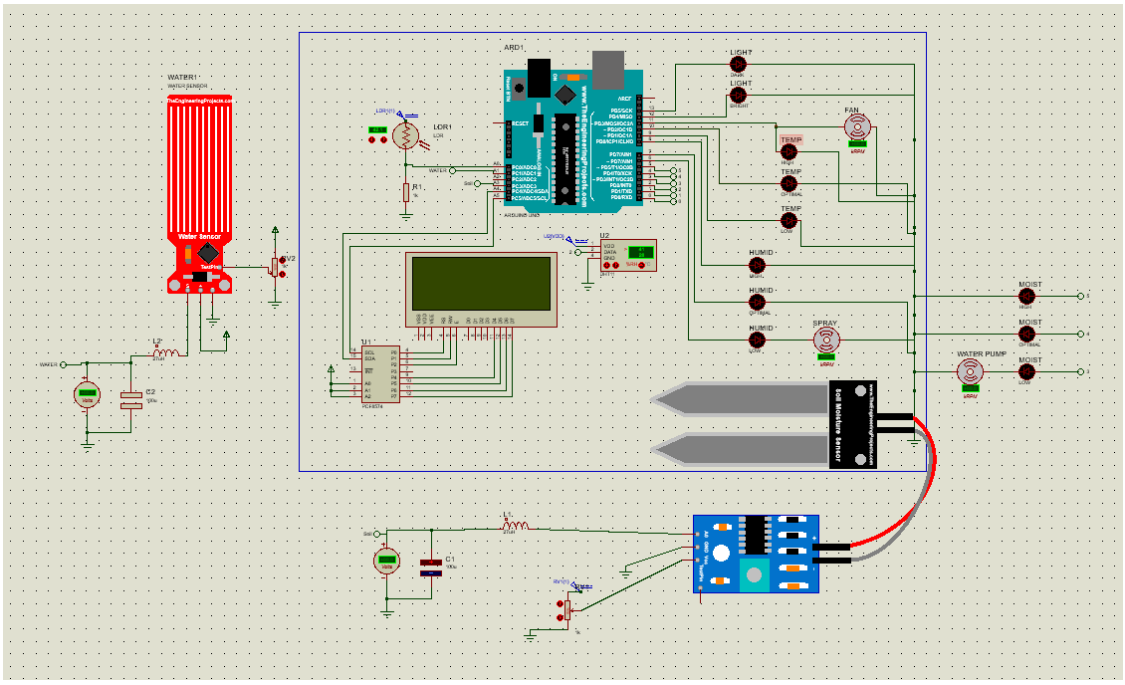


Figure 5: Proteus schematic of the integrated system: Arduino Uno with LDR, DHT11, water-level sensor, soil-moisture sensor, I^{2}C LCD, status LEDs and fan, spray and pump motor models.



Figure 6: Hardware test: high humidity → fan on (left), dark → light on (centre), tank level 0 % (right).

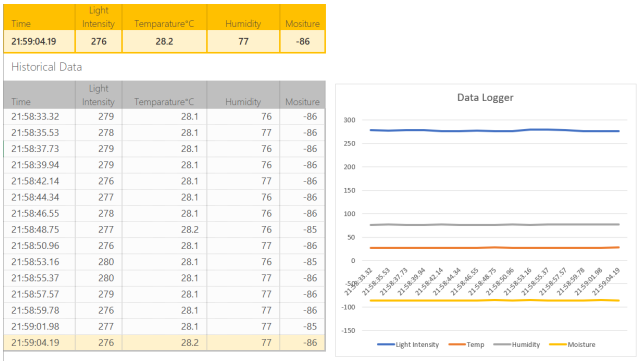


Figure 7: Serial data log of the system in Excel Data Streamer.

- Range checks written as `150 > x < 200` are not valid comparisons in C. They give the intended result only because they are reached after the other two branches.
- Two indicator LEDs use D0/D1, which are shared with the serial port used for logging.
- A failed DHT reading (NaN) falls through to the "optimal" branch instead of being reported as an error.
- The `delay(2000)` per screen makes one loop pass take several seconds, so the system reacts slowly.

In addition, the lamp, fan, spray and pump were represented

by LEDs and motor models, and no mains-powered actuators were connected.

VI. CONCLUSION AND FUTURE WORK

An Arduino-based system for monitoring and controlling the growing conditions of a plant was designed, simulated in Proteus and tested on hardware. The light and humidity control behaved as specified in both the simulation and the hardware test, and the LCD interface and serial logging worked as intended. The firmware review identified concrete defects in the integrated version. The next steps are:

- fix these defects and add hysteresis and a dry-run interlock for the pump;
- replace the blocking delays with `millis()`-based timing;
- drive real actuators through relays and dim the lamp with PWM;
- add a GSM module for remote refill alerts, as proposed in the original design.

CONTRIBUTIONS

M. A. Sadiq: light, humidity, water-level and soil-moisture subsystems; integration and Proteus simulation of the complete system (with Nabeel); hardware test and data logging. Nabeel: temperature subsystem; integration and simulation (with M. A. Sadiq). Shimal: project documentation. The Proteus models of the Arduino Uno, water sensor and soil-moisture sensor are from TheEngineeringProjects.

REFERENCES

[1] Microchip Technology, *ATmega328P Datasheet*. [Online]. Available: `https://www.microchip.com/en-us/product/ATmega328P`

[2] Arduino, *Arduino UNO R3 Documentation*. [Online]. Available: `https://docs.arduino.cc/hardware/uno-rev3`

[3] Aosong Electronics, *DHT11 Humidity & Temperature Sensor Datasheet*.

[4] Hitachi, *HD44780U (LCD-II) Dot Matrix Liquid Crystal Display Controller/Driver Datasheet*.

[5] Texas Instruments, *PCF8574 Remote 8-Bit I/O Expander for I^2C Bus Datasheet*.

[6] Labcenter Electronics, *Proteus VSM Simulation*. [Online]. Available: `https://www.labcenter.com/simulation`

[7] Adafruit Industries, *DHT Sensor Library*. [Online]. Available: `https://github.com/adafruit/DHT-sensor-library`

[8] F. de Brabander *et al.*, *LiquidCrystal_I2C Library*. [Online]. Available: `https://github.com/johnrickman/LiquidCrystal_I2C`